Trabajo Práctico Unidad 6

Mantenimiento, respaldo y recuperación de VoltTech

Informe de lo realizado en cada actividad · PostgreSQL 17.10 · Base volttech_u6

Alumno	Joaquin Antonio Mendez Reynoso	**Materia**	Bases de Datos II · Comisión 4
Motor	PostgreSQL 17.10 (localhost:5432)	**Base principal**	volttech_u6
Repositorio	https://github.com/VIKROOM/db_Mendez_COM4_U6.git	**Fecha**	10 de noviembre de 2026

Resumen del trabajo

El TP se resolvió sobre una instancia real de PostgreSQL siguiendo las cinco actividades de la consigna: primero la especificación y el plan, después el respaldo, luego el monitoreo y el mantenimiento, a continuación el simulacro de desastre contra reloj y, por último, el informe post-incidente con la verificación del trabajo asistido por OpenCode. La base es la aplicación VoltTech del equipo (comercio de electrónica) cargada con volúmenes reales: 50.002 productos, 20.001 clientes, 200.001 pedidos y 800.002 líneas de pedido. Cada apartado de este documento indica qué pedía la actividad, qué se hizo, cuál fue el resultado medido y en qué archivo se puede auditar.

Las cinco actividades y su resultado

#	Actividad de la consigna	Entregable	Resultado
1	Parte 1 · Especificación y plan de mantenimiento	SPEC_U6.md	Plan de 4 tareas con frecuencia y evidencia (37 líneas)
2	Parte 2 · Respaldo automatizable	backup_volttech.ps1	10,81 MB · 58 objetos · SHA-256 · prueba de falla
3	Parte 3 · Monitoreo y mantenimiento	health_check.sql + maintenance.sql	6 bloques de control · 5.000 tuplas recuperadas
4	Parte 4 · Simulacro de desastre contra reloj	restore_test.ps1	RTO 7,60 s · RPO 58 s · 7/7 conteos
5	Parte 5 · Informe post-incidente y uso de IA	informe_u6.pdf + BITACORA_IA.md	1 página · 102 PASS / 0 FAIL

7,60 s	58 s	7/7	102 / 0
RTO observado (meta ≤ 20 min)	RPO observado (meta ≤ 24 h)	Conteos validados contra el snapshot	Autoevaluación PASS / FAIL

El repositorio con todos los archivos, scripts y evidencias es https://github.com/VIKROOM/db_Mendez_COM4_U6.git. Las salidas crudas están en **evidencias/**; los scripts pueden volver a ejecutarse siguiendo el Anexo B.

1 Especificación y plan de mantenimiento

Parte 1 de la consigna — COMPLETADA

Consigna. Crear SPEC_U6.md (máximo una página) indicando qué se automatiza, con qué frecuencia y qué condición confirma que cada tarea terminó correctamente.

Qué se hizo.

- Se redactó **SPEC_U6.md** en 37 líneas con alcance, tabla de tareas, metas de RPO/RTO, reglas de seguridad y criterios de aceptación.
- Se definieron las cuatro tareas mínimas con frecuencia, identidad, condición de éxito y evidencia verificable; no se automatizan decisiones de intervención, solo la observación y la ejecución segura.
- Se fijaron las metas del laboratorio: **RPO ≤ 24 h** y **RTO ≤ 20 min**, con medición real y no estimada. Quedaron fuera de alcance el respaldo físico, PITR y la replicación.

Tarea mínima	Frecuencia	Condición que confirma el éxito	Evidencia
Respaldo de volttech_u6	Diaria, 02:00	pg_dump termina en 0, el dump se lista, se calcula SHA-256 y se registra en bitácora.	.dump fechado + log + .sha256
Chequeo de salud	Diaria, inicio de jornada	Devuelve tamaño, actividad de conexiones, estado de tablas y consultas activas.	Salida de health_check.sql
Revisión de mantenimiento	Semanal	Se observan tuplas muertas y autovacuum; solo se interviene si se supera el umbral.	Salida antes/después + bitácora
Restauración de prueba	Semanal	volttech_u6_restore_test se crea, restaura y supera conteos e invalid=0.	Log + métricas RTO/RPO

Reglas de seguridad que gobiernan todo el procedimiento

#	Regla de seguridad no negociable
1	volttech_u6 no se elimina, reemplaza ni se restaura durante el simulacro.
2	Toda operación destructiva se limita a volttech_u6_restore_test; el script aborta si el destino no termina en _restore_test.
3	Ninguna contraseña se escribe en scripts, SQL, logs ni repositorio: se usa %APPDATA%\postgresql\pgpass.conf.
4	Roles de mínimo privilegio: mant_backup (lectura), mant_monitor (pg_monitor) y mant_restore (CREATEDB + lectura).
5	VACUUM FULL queda prohibido como mantenimiento periódico; solo se admite VACUUM (ANALYZE) justificado.
6	Todo script aborta ante el primer error y deja un log con fecha, operación, resultado y código de salida.
7	Un respaldo no se considera válido hasta que se restaura y se validan cantidades y reglas lógicas.

Cómo se verificó La especificación se usó como contrato: soporte/verificar_entrega.ps1 comprueba automáticamente que mencione las frecuencias, RPO/RTO, la base descartable, pgpass.conf y la prohibición de VACUUM FULL. Los controles de plan quedaron en PASS.

Evidencias: SPEC_U6.md · evidencias/verificacion_entrega.txt (controles de plan).

2 Respaldo automatizable

Parte 2 de la consigna — COMPLETADA

Consigna. Crear backup_volttech.ps1 para generar un respaldo custom de volttech_u6, con fecha y hora en el nombre, registro en logs, error si pg_dump falla y sin contraseñas embebidas.

Qué se hizo.

- El script toma un **snapshot de conteos antes** del respaldo y ejecuta **pg_dump -Fc -Z 6 -w** con el rol **mant_backup**, escribiendo primero un archivo **.parcial**.
- Si pg_dump falla: aborta con código distinto de cero, elimina el **.parcial** y registra **ERROR** en el log y en la bitácora; nunca deja un archivo incompleto con nombre válido.
- Si termina bien: verifica el dump con **pg_restore --list**, lo renombra con fecha y hora, calcula **SHA-256** y aplica retención solo a sus propios archivos (patrón volttech_u6_*.dump).
- Se ejecutó manualmente antes de considerarlo apto y se probó el camino de error con una tabla inexistente.

Control	Valor observado
Archivo	volttech_u6_20261110_203628.dump (formato custom -Fc, compresión -Z 6)
Tamaño y objetos	11.332.222 bytes (10,81 MB) · 58 entradas en el TOC
Duración	2,9 (20:36:28–20:36:31)
Integridad	SHA-256 621D798175B2077F…64E92C13
Bitácora	Fila OK registrada en mantenimiento.bitacora
Retención	14 días; 0 archivos eliminados en esta corrida

Prueba de falla ejecutada. Con `-PgDumpExtra '--table=public.tabla_que_no_existe'`, pg_dump terminó con **exit ?**, el log quedó en **ERROR**, se eliminó el .parcial y no apareció ningún .dump válido.

Problema real y corrección

El respaldo terminó bien en la primera ejecución porque los errores típicos se cubrieron en el diseño: el INSERT en bitácora se escribe mediante un archivo temporal con psql -f (psql no interpola :'variables' en -c), pg_dump corre con -w y pgpass.conf (sin contraseñas embebidas ni esperas en tareas programadas) y el dump se graba primero como .parcial y se renombra recién tras pasar pg_restore --list. La prueba de falla confirmó ese camino de error: exit 1, .parcial eliminado y ERROR en bitácora.

Evidencias: evidencias/backup_ejecucion.log · evidencias/backup_listado.txt · evidencias/backup_sha256.txt · evidencias/fallo_pg_dump.log.

3 Monitoreo y mantenimiento

Parte 3 de la consigna

COMPLETADA

Consigna. Crear health_check.sql con al menos cuatro controles (tamaño, conexiones, estado de tablas y consultas activas) y maintenance.sql con revisión de autovacuum/ANALYZE más una acción segura y justificada, sin VACUUM FULL periódico.

Qué se hizo.

- **Monitoreo:** health_check.sql cubre seis bloques: tamaño de base y tablas; conexiones, commits/rollbacks, cache hit, deadlocks y temporales; tuplas vivas/muertas, vacuum y analyze; consultas activas, idle in transaction y bloqueos; edad del XID (wraparound); y antigüedad del último respaldo OK para vigilar el RPO.
- **Observación previa:** maintenance.sql revisa los parámetros de autovacuum, overrides por tabla, transacciones largas y estadísticas por tabla antes de decidir.
- **Umbral documentado:** se interviene si una tabla tiene ≥ 5 % de tuplas muertas y ≥ 1.000 filas, o ≥ 10.000 tuplas muertas. Solo **producto** calificó, con 5000 muertas sobre 50002 (9,09 %), generadas por la simulación de actividad (UPDATE del 10 % del stock).
- **Acción segura:** se ejecutó **VACUUM (ANALYZE, VERBOSE)** solo sobre esa tabla mediante \gexec: quedaron 0 tuplas muertas y las estadísticas actualizadas. No se usó VACUUM FULL.

Parámetro observado	Valor	Lectura
autovacuum	on	El proceso automático está habilitado.
autovacuum_vacuum_scale_factor	0,2	Umbral relativo alto: con tablas grandes tarda en actuar.
autovacuum_analyze_scale_factor	0,1	Estadísticas se actualizan al 10 % de cambios.
autovacuum_vacuum_threshold	50	Margen fijo adicional de tuplas muertas.
autovacuum_naptime	60 s	Frecuencia de revisión del proceso.
autovacuum_max_workers	3	Cantidad de workers disponibles.

Indicador	Antes	Después
Tuplas vivas en producto	50002	50002
Tuplas muertas en producto	5000	0
Porcentaje de tuplas muertas	9,09 %	0,00 %
last_vacuum	(null)	2026-11-10 20:35:44.130101-03
last_analyze	2026-11-10 20:22:47.085465-03	2026-11-10 20:35:44.523254-03
Tamaño total de producto	14 MB	14 MB (sin reescritura)

Corrección de criterio

Un VACUUM FULL semanal “para recuperar espacio” fue descartado desde el diseño: bloquea la tabla con ACCESS EXCLUSIVE y la consigna lo prohíbe como mantenimiento periódico. En su lugar, maintenance.sql observa primero y ejecuta VACUUM (ANALYZE) solo en las tablas que superan el umbral.

Evidencias: health_check.sql · maintenance.sql · evidencias/health_check_antes.txt · evidencias/health_check_despues.txt · evidencias/maintenance_salida.txt.

4 Simulacro de desastre contra reloj

Parte 4 de la consigna

COMPLETADA

Consigna. Restaurar el último respaldo en volttech_u6_restore_test, cronometrar desde antes de createdb hasta terminar las validaciones, comparar conteos con el snapshot previo y calcular RTO y RPO observados.

Qué se hizo.

- El script valida el dump con **pg_restore --list**, verifica el **SHA-256** y ejecuta un **preflight** de conexiones antes de crear la base. Aborta si el destino es volttech_u6 o no termina en **_restore_test**.
- Arranca el cronómetro inmediatamente antes de **createdb** y restaura con **pg_restore --no-owner --no-privileges --exit-on-error -j 4** usando **mant_restore** (CREATEDB, sin superusuario).
- Detiene el cronómetro al terminar la última validación. El RPO se calcula como la antigüedad del .dump respecto del inicio del simulacro.
- volttech_u6 nunca se toca: el simulacro corre de principio a fin sobre **volttech_u6_restore_test**, que queda conservada para inspección junto con la vista materializada repoblada.

Métrica / control	Previsto	Observado	Cumple
RPO (pérdida máxima)	≤ 24 h (86.400 s)	58 s (0 min 58 s) de antigüedad	Sí
RTO (recuperación)	≤ 20 min (1.200 s)	7,60 s	Sí
Conteos vs. snapshot	7/7 coincidentes	7/7 coincidentes	Sí
Productos / detalles inválidos	0 / 0	0 / 0	Sí
Huérfanos referenciales	0	0	Sí
Restricciones restauradas	16	16	Sí
Vista detalle_pedido	Presente	800.002 filas	Sí

Conteos validados: 2 categorías, 20.001 clientes, 50.002 productos, 200.001 pedidos, 800.002 líneas, suma de cantidades 1.599.736 e importe 800.285.374.188,00.

Objetos de la app restaurados: vistas v_productos_vigentes, v_pedidos_cliente y v_detalle_pedido_producto; vista materializada mv_facturacion_categoria_mes repoblada con REFRESH; procedimientos registrar_pedido y ajustar_stock; extensión pg_trgm y su índice gin sobre producto.nombre.

Problema real y corrección

pg_dump restaura las vistas materializadas sin datos: mv_facturacion_categoria_mes quedaba vacía en volttech_u6_restore_test tras un pg_restore impecable (0 errores y 7/7 conteos). No era un fallo del respaldo sino un comportamiento estándar que el procedimiento debía cubrir: validando solo conteos de tablas, la base habría pasado el chequeo con la vista de reportes vacía. Se agregó REFRESH MATERIALIZED VIEW a soporte/validaciones_restauracion.sql y quedó como evidencia: 26 filas tras repoblar (restore_validaciones.txt, sección 8).

Evidencias: evidencias/restore_ejecucion.log · evidencias/restore_metrics.json · evidencias/restore_validaciones.txt.

5 Informe post-incidente y uso de IA

Parte 5 de la consigna — COMPLETADA

Consigna. Entregar un informe de una página con fecha y hora del respaldo, tiempo de restauración, RPO/RTO previstos y observados, validaciones, problema real, mejora concreta e indicación del trabajo asistido por OpenCode.

Qué se hizo.

- Se redactó **informe_u6.pdf**, de una página, con los siete puntos exigidos y el link del repositorio.
- El uso de OpenCode quedó registrado en BITACORA_IA.md: 12 decisiones de la IA revisadas por el operador, incluidas las propuestas descartadas y por qué (credenciales reemplazadas por pgpass, rechazo de VACUUM FULL, vista de compatibilidad detalle_pedido, límite de conexiones para pg_restore -j 4 y repoblado de la vista materializada).
- Se ejecutó **soporte/verificar_entrega.ps1** contra la entrega completa: **102 PASS / 0 FAIL**, incluyendo archivos, ausencia de contraseñas, ausencia de VACUUM FULL, evidencias y métricas del simulacro.

Requisito del informe de una página	Dónde se cumple
Fecha y hora del respaldo utilizado	informe_u6.pdf §1: 2026-11-10 (20:36:28–20:36:31)
Tiempo total de restauración y validación	§2: RTO 7,60 s
RPO y RTO previstos y observados	§2: ≤ 24 h / 58 s y ≤ 20 min / 7,60 s
Validaciones de tablas y restricciones lógicas	§3: 7/7 conteos, invalid=0, huérfanos=0, 16 restricciones y objetos VoltTech
Problema real encontrado	§4 (detallado también en BITACORA_IA.md)
Mejora concreta para la siguiente ejecución	§5
Uso de OpenCode y verificación	§6 y BITACORA_IA.md; autoevaluación 102 PASS / 0 FAIL

Ejemplos de revisión humana sobre lo propuesto por la IA

Propuesta inicial de la IA	Revisión humana y corrección aplicada
Ejecutar respaldo, monitoreo y restauración con el superusuario postgres	Rechazada: roles de mínimo privilegio (mant_backup, mant_monitor, mant_restore); la credencial del administrador se usó solo en el setup inicial y nunca quedó en archivos.
VACUUM FULL semanal para recuperar espacio en disco	Rechazada: bloquea la tabla con ACCESS EXCLUSIVE y la consigna lo prohíbe como mantenimiento periódico; se aplicó VACUUM (ANALYZE) solo a las tablas sobre el umbral de 5 % y 1.000 filas o 10.000 muertas.
Renombrar linea_pedido como detalle_pedido para cumplir la consigna	Corregida: se creó la vista de compatibilidad public.detalle_pedido sobre la tabla real, sin alterar el DDL del proyecto VoltTech.
CONNECTION LIMIT 3 para el rol de restauración	Ajustada a 10 con preflight: pg_restore -j 4 puede abrir hasta 5 conexiones simultáneas y un límite menor abortaría el simulacro a mitad de restauración.
Aceptar la vista materializada tal como la restaura pg_dump	Corregida: pg_dump la restaura sin datos; se agregó REFRESH MATERIALIZED VIEW a las validaciones y quedó repoblada con 26 filas.

Mejora aplicada Para la siguiente ejecución: (1) programar backup_volttech.ps1 y health_check.sql en el Programador de tareas de Windows (02:00) para que el RPO de 24 h no dependa de la ejecución manual, y (2) convertir la sección 6 del chequeo de salud (antigüedad del último respaldo OK) en una alerta explícita cuando supere las 20 h, margen sobre el RPO; hoy solo informa la antigüedad.

Evidencias: informe_u6.pdf · BITACORA_IA.md · evidencias/verificacion_entrega.txt.

Cumplimiento de entregables y criterios

La tabla siguiente contrasta lo pedido por la cátedra con lo efectivamente entregado y verificado.

Entregable de la consigna	Estado	Ubicación
SPEC_U6.md	Completo	SPEC_U6.md (37 líneas)
backup_volttech.ps1 o .sh	Completo y probado	backup_volttech.ps1
health_check.sql	Completo (4+ controles)	health_check.sql (6 bloques)
maintenance.sql	Completo y justificado	maintenance.sql
restore_test.ps1 o .sh (opcional)	Completo	restore_test.ps1
informe_u6.pdf	Completo (1 página)	informe_u6.pdf
Evidencias y bitácora	Completo	evidencias/, logs/, BITACORA_IA.md

Criterios de evaluación de la consigna

Criterio de evaluación	Peso	Cómo se cubrió
Especificación clara y plan coherente	2,0	SPEC de 37 líneas con tareas, frecuencia, RPO/RTO y reglas.
Respaldo automatizable, seguro y probado	2,0	pg_dump -Fc, SHA-256, logs, bitácora y prueba de falla.
Monitoreo y mantenimiento correctos	2,0	Seis bloques de control y VACUUM (ANALYZE) justificado.
Restauración, validaciones y RPO/RTO	3,0	7,60 s y 58 s; 7/7 conteos; invalid=0 y huérfanos=0.
Informe breve, evidencias y verificación de IA	1,0	Informe de 1 página, BITACORA_IA y 102 PASS / 0 FAIL.

Conclusión. Las cinco actividades quedaron completas y verificables: plan escrito, respaldo con integridad comprobada y prueba de falla, monitoreo con seis bloques, mantenimiento decidido por métricas, restauración real en 7,60 s con validación 7/7 y un informe de una página con la intervención de OpenCode documentada. La base volttech_u6 nunca se tocó y ninguna credencial quedó en el repositorio. La autoevaluación cerró con 102 PASS / 0 FAIL.

Repositorio: https://github.com/VIKROOM/db_Mendez_COM4_U6.git · el informe de una página se mantiene como entrega oficial de la Parte 5.

Anexo A · Índice de evidencias

Cada resultado del informe se puede auditar en el archivo correspondiente dentro de la entrega.

Archivo	Qué demuestra
evidencias/backup_ejecucion.log	Respaldo OK: 11.332.222 bytes, 58 objetos, SHA-256 y bitácora.
evidencias/backup_listado.txt	El dump se puede leer con pg_restore --list.
evidencias/backup_sha256.txt	Huella SHA-256 del archivo de respaldo.
evidencias/fallo_pg_dump.log	Prueba de falla: exit ?, ERROR y sin .dump válido.
evidencias/health_check_antes.txt	Estado previo: producto con 5000 tuplas muertas (9,09 %).
evidencias/health_check_despues.txt	Estado posterior: 0 tuplas muertas y ANALYZE actualizado.
evidencias/maintenance_salida.txt	Observación de autovacuum y VACUUM (ANALYZE) justificado.
evidencias/restore_ejecucion.log	Simulacro completo con preflight y RTO/RPO observados.
evidencias/restore_metrics.json	Métricas finales: RTO 7,60 s, RPO 58 s, validaciones OK.
evidencias/restore_validaciones.txt	7/7 conteos, invalid=0, huérfanos=0, 16 restricciones y objetos VoltTech.
evidencias/counts_pre_backup.txt	Snapshot de conteos previo al respaldo.
evidencias/verificacion_entrega.txt	Autoevaluación: 102 PASS / 0 FAIL.

Anexo B · Reproducción

Los comandos se ejecutan desde la raíz de la carpeta del TP. Requieren PostgreSQL 16+ y pgpass.conf configurado.

Paso	Comando
Respaldo diario	`.\backup_volttech.ps1`
Salud antes/después	`psql -h localhost -U mant_monitor -d volttech_u6 -X -f .\health_check.sql`
Mantenimiento	`psql -h localhost -U postgres -d volttech_u6 -X -f .\maintenance.sql`
Simulacro	`.\restore_test.ps1 -Force`
Autoevaluación	`.\soporte\verificar_entrega.ps1`

Repositorio: https://github.com/VIKROOM/db_Mendez_COM4_U6.git · Informe oficial de la Parte 5: informe_u6.pdf (1 página).